Механизм блокирования

- Высокоуровневый параллелизм при работе с данными:
  - блокировки на уровне строки для операций вставки, обновления или удаления;
  - для запросов блокировки не требуются.
- Автоматическое управление очередью.
- Блокировка сохраняется до завершения транзакции (с помощью оператора **COMMIT** или **ROLLBACK**).

Транзакция 1

Транзакция 2

```
SQL> UPDATE employees
  2  SET salary=salary+100
  3  WHERE employee_id=100;
```

```
SQL> UPDATE employees
  2  SET salary=salary*1.1
  3  WHERE employee_id=101;
```

ORACLE

© Oracle, 2007. Все права защищены.

Механизм блокирования

Механизм блокирования разработан для обеспечения максимально возможной степени параллельного доступа к данным базы данных. Для транзакций, которые изменяют данные, используются блокировки на уровне строки, а не на уровне блока или таблицы. При изменениях объектов (например, перемещении таблицы) возникают блокировки на уровне объекта, при этом база данных или схема не блокируются.

При запросе данных блокировки не возникают. Кроме того, запрос будет выполнен успешно даже в том случае, если опрашиваемые данные уже заблокированы (будут отображены первоначальные значения, восстановленные из данных отмены операций и существовавшие до создания блокировки).

В ситуациях, когда несколько транзакций пытаются заблокировать один ресурс, блокировку создаст первая транзакция, которая попыталась обратиться к нему. Другие транзакции будут ожидать окончания первой транзакции. Механизм обслуживания очередей работает автоматически и не требует вмешательства администратора.

Все автоматические блокировки снимаются после подтверждения транзакции. Транзакция считается защищенной после выполнения оператора `COMMIT` или `ROLLBACK`. Если транзакцию выполнить не удалось, тот же фоновый процесс автоматически отменит все изменения, внесенные незаконченной транзакцией, и снимет все созданные ею блокировки.

Параллелизм при работе с данными

Время:	Транзакция 1	UPDATE hr.employees SET salary=salary+100 WHERE employee_id=100;
	Транзакция 2	UPDATE hr.employees SET salary=salary+100 WHERE employee_id=101;
09:00:00	Транзакция 3	UPDATE hr.employees SET salary=salary+100 WHERE employee_id=102;
	...	...
	Транзакция *x*	UPDATE hr.employees SET salary=salary+100 WHERE employee_id=*xxx*;

ORACLE

© Oracle, 2007. Все права защищены.

Параллелизм при работе с данными

Механизм блокирования по умолчанию хорошо гранулирован и применяется на уровне строк. Разные транзакции могут обновлять различные строки одной таблицы, не влияя друг на друга.

По умолчанию блокирование осуществляется на уровне строки, однако база данных Oracle поддерживает ручные блокировки и на более высоких уровнях, если в них возникнет необходимость:

```
SQL> LOCK TABLE employees IN EXCLUSIVE MODE;
Таблицы Блокированы.
```

Согласно вышеприведенному примеру, любая транзакция, которая попытается обновить строку в заблокированной таблице, будет вынуждена дождаться завершения транзакции, создавшей блокировку. `EXCLUSIVE` является самым строгим режимом блокирования.

Существуют и другие режимы блокирования:

- **`ROW SHARE`:** делает возможным параллельный доступ к данным заблокированной таблицы и не дает сеансам заблокировать всю таблицу для монопольного доступа.
- **`ROW EXCLUSIVE`:** этот режим подобен режиму `ROW SHARE`, однако запрещает блокирование в режиме `SHARE`. Блокировки типа `ROW EXCLUSIVE` создаются автоматически при обновлении, вставке или удалении данных. Блокировки типа `ROW EXCLUSIVE` разрешают работу множеству процессов чтения и одному процессу записи.

- **SHARE:** разрешает параллельные очереди, но запрещает обновления заблокированной таблицы. Блокировка `SHARE` необходима (и запрашивается автоматически) для создания индекса таблицы. Для оперативного создания индекса требуется блокировка `ROW SHARE`, которая используется при построении индекса.

Параллелизм при работе с данными (продолжение)

Разделяемые блокировки позволяют работать с данными нескольким процессам чтения, а процессам записи запрещают. Разделяемые блокировки используются прозрачно при удалении или обновлении строк в родительской таблице, которая имеет дочернюю таблицу с ограничениями внешнего ключа для родительской таблицы.

- **SHARE ROW EXCLUSIVE:** используется для опроса всей таблицы и позволяет другим опрашивать строки таблицы, но при этом запрещает им блокировать таблицу в режиме SHARE или обновлять строки.
- **EXCLUSIVE:** разрешает опрашивать заблокированную таблицу и запрещает любые манипуляции над ней. Блокировка EXCLUSIVE необходима при удалении таблицы.

Подобно любому запросу на блокировку, операторы ручной блокировки ожидают, пока все сеансы, уже имеющие блокировки или создавшие запрос на блокировку, снимут свои блокировки. Команда LOCK может принимать особый аргумент, который управляет режимом ожидания NOWAIT.

NOWAIT немедленно возвращает контроль над указанной таблицей, если она уже была заблокирована другим сеансом:

```
SQL> LOCK TABLE hr.employees IN SHARE MODE NOWAIT;
LOCK TABLE hr.employees IN SHARE MODE NOWAIT
              *
ERROR at line 1:
ORA-00054: resource busy and acquire with NOWAIT specified
```

Обычно не возникает необходимости в ручной блокировке объектов. Механизм автоматического блокирования обеспечивает параллельный доступ к данным для большинства приложений. Oracle рекомендует избегать применения ручных блокировок, особенно при разработке приложений. Неадекватное использование высоких уровней блокирования может негативно сказаться на производительности.

DML-блокировки

Транзакция 1

```
SQL> UPDATE employees
  2  SET salary=salary*1.1
  3  WHERE employee_id= 107;
1 row updated.
```

Транзакция 2

```
SQL> UPDATE employees
  2  SET salary=salary*1.1
  3  WHERE employee_id= 106;
1 row updated.
```

Каждая DML-транзакция должна получить *две* блокировки:

- блокировку **EXCLUSIVE** на уровне строки для обновляемой строки или строк;
- блокировку **ROW EXCLUSIVE** на уровне таблицы для таблицы, содержащей строки.

ORACLE

© Oracle, 2007. Все права защищены.

DML-блокировки

Каждая DML-транзакция использует *две* блокировки:

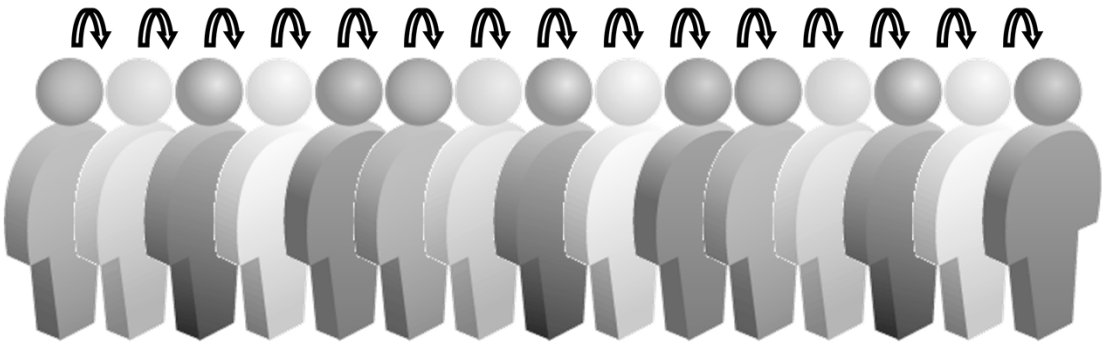
- блокировку EXCLUSIVE на уровне строки для обновляемой строки или строк
- блокировку ROW EXCLUSIVE на уровне таблицы для таблицы, содержащей строки
  Это не даст другому сеансу заблокировать всю таблицу (возможно, для ее удаления или усечения) при внесении изменений.

Блокировка ROW EXCLUSIVE на уровне таблицы не позволяет DDL изменять метаданные словаря при обработке неподтвержденной транзакции. Это позволяет поддерживать целостность словаря и согласованность данных на протяжении жизненного цикла транзакции.

Механизм постановки в очередь

Механизм постановки в очередь отслеживает:

- сеансы, ожидающие блокировки;
- запрашиваемый режим блокирования;
- порядок, в котором сеансы запрашивали блокировки.



ORACLE

9 - 5 © Oracle, 2007. Все права защищены.

Механизм обслуживания очереди

Очередь запросов на блокировки обслуживается автоматически. После завершения транзакции, создавшей блокировку, следующий в очереди сеанс получает право на создание блокировки.

Механизм обслуживания очереди отслеживает порядок, в котором поступали запросы на создание блокировок, а также запрашиваемые режимы блокировок.

Сеанс, который уже создал блокировку, может создать запрос на *преобразование* блокировки, не переходя в конец очереди. Например, сеанс создал блокировку `SHARE` на уровне таблицы. Сеанс может создать запрос на преобразование блокировки `SHARE` в блокировку типа `EXCLUSIVE`. Если таблица еще не имеет блокировок типа `EXCLUSIVE` или `SHARE`, то сеансу, который создал блокировку `SHARE`, будет предоставлено право на создание блокировки `EXCLUSIVE` вне очереди.

Примечание. Существует две категории участников очереди: участники без разделяемого владения и участники с разделяемым владением, которые не поднимают уровень блокировки. Участники второй категории известны как *конверторы*. Они всегда имеют более высокий приоритет, чем остальные участники очереди, даже если они ожидают меньшее количество времени.

Конфликты блокировок

Транзакция 1	Время	Транзакция 2
`UPDATE employees SET salary=salary+100 WHERE employee_id=100;` `1 row updated.`	9:00:00	`UPDATE employees SET salary=salary+100 WHERE employee_id=101;` `1 row updated.`

`UPDATE employees SET COMMISION_PCT=2 WHERE employee_id=101;`
Сеанс ожидает постановки в очередь из-за конфликта блокировок.
	9:00:05	`SELECT sum(salary) FROM employees;`
		`SUM(SALARY)`
		`----------`
		`692634`

Сеанс все еще ожидает!	16:30:00	Множество операций выборки, вставки, обновления и удаления за последние 7,5 часов, но без каких-либо фиксаций или откатов!
`1 row updated.` Сеанс продолжается.	16:30:01	`commit;`

ORACLE

© Oracle, 2007. Все права защищены.

Конфликты блокировок

Конфликты блокировок обычно устраняются временем и механизмом обслуживания очереди. В некоторых редких случаях для устранения конфликта блокировок может потребоваться вмешательство администратора. В примере, представленном на втором слайде, вторая транзакция создает блокировку для одной строки в 9:00:00 и при этом не подтверждает изменения, из-за чего блокировка не снимается. Первая транзакция пытается обновить всю таблицу в 9:00:05, для чего необходимо создать блокировки для всех строк. Первая транзакция заблокирована второй транзакцией, пока та не подтвердила изменения в 16:30:01.

Пользователь, пытаясь осуществить первую транзакцию, почти наверняка свяжется с администратором, который должен выяснить причину конфликта и устранить ее.

Возможные причины конфликтов блокировок

- Неподтвержденные изменения.
- Транзакции, которые обрабатываются длительное время.
- Неадекватно высокие уровни блокирования.

ORACLE

9 - 7 © Oracle, 2007. Все права защищены.

Возможные причины конфликтов блокировок

Наиболее распространенной причиной возникновения конфликта блокировок являются неподтвержденные изменения, однако существуют и другие причины:

- **Транзакции, которые выполняются длительное время:** множество приложений используют пакетную обработку для выполнения массовых обновлений. Такие операции обычно выполняются по расписанию в интервалы времени с низкой активностью пользователей или при ее отсутствии, но в некоторых случаях операция может не завершиться вовремя, или для ее завершения может потребоваться значительно больше времени, чем было рассчитано. Конфликты блокировок довольно часто возникают при одновременном выполнении операций пакетной обработки.
- **Неадекватно высокие уровни блокирования:** не все базы данных поддерживают низкоуровневое блокирование (Oracle получил поддержку этой функции в 6 версии, которая вышла в 1988 году). Некоторые базы данных все еще осуществляют блокировки на уровне страницы или таблицы. Разработчики приложения обычно используют при создании приложений множество различных баз данных с искусственно завышенными уровнями блокирования, из-за чего база данных Oracle вынуждена делать то же самое для обеспечения совместимости с менее гибкими системами баз данных. Разработчики, которые не имеют достаточного опыта работы с базами данных Oracle, также иногда используют без необходимости более высокие уровни блокирования, чем необходимо.

Выявление конфликтов блокировок

Выберите «Blocking Sessions» (Блокирующие сеансы) на странице «Performance» (Производительность).

Blocking Sessions

Page Refreshed Jul 31, 2007 1:01:18 PM PDT Refresh

View Session | Kill Session

Expand All | Collapse All

Select	Username	Sessions Blocked	Session ID	Serial Number	SQL ID	Wait Class	Wait Event	P1 Value	P2 Value	P3 Value	Seconds in Wait
	Blocking Sessions										
	NGREENBERG	1	170	17463		Idle	SQL*Net message from client	1650815232	1	0	90
	SMAVRIS	0	124	20568	6smgtv6h8958b	Application	enq: TX - row lock contention	1415053318	458762	691	39

Additional Monitoring Links

Щелкните ссылку «Идентификатор сеанса», чтобы просмотреть сведения о блокировках сеансов, включая текущий SQL-оператор.

ORACLE

9 - 8 © Oracle, 2007. Все права защищены.

Выявление конфликтов блокировок

Для выявления конфликтов блокировок воспользуйтесь страницей «Blocking Sessions» (Блокирующие сеансы) в Enterprise Manager. Конфликтующие запросы на создание блокировок будут отображены в иерархическом порядке со сведениями о сеансе, создавшем блокировку (вверху) и сеансах, которые создали запросы на блокировки (внизу).

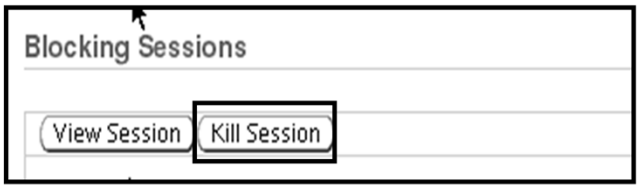
Для каждого сеанса, участвующего в конфликте, будет отображено имя пользователя, идентификатор сеанса и время ожидания в секундах. Перейдите к идентификатору сеанса, чтобы отобразить операторы SQL, которые выполняются или были вызваны сеансом.

Автоматический монитор диагностики базы данных (Automatic Database Diagnostic Monitor – ADDM) автоматически выявляет конфликты блокировок и дает рекомендации относительно неэффективных приемов блокирования.

Устранение конфликтов блокировок

Для разрешения конфликта блокировок:

- сеанс, создавший блокировку, должен подтвердить или отменить изменения;
- следует прервать сеанс, создавший блокировку (при крайней необходимости).



ORACLE

9 - 9 © Oracle, 2007. Все права защищены.

Устранение конфликтов блокировок

Чтобы устранить конфликт блокировок, сеанс должен освободить созданную блокировку. Наилучший способ достичь этого – связаться с пользователем и попросить его завершить транзакцию.

В крайнем случае администратор может прервать сеанс, создавший блокировку, нажав кнопку «Kill Session» (Удалить сеанс). Помните, что при таком завершении сеанса все изменения, внесенные данной транзакцией, будут потеряны (будет выполнен откат). Пользователю завершенного сеанса нужно будет повторно подключиться к базе данных и повторить все операции, которые были выполнены после последней операции подтверждения.

Пользователи, сеанс которых был прерван, получат следующее сообщение об ошибке при попытке выполнить новый SQL-оператор:

```
ORA-03135: connection lost contact
```

Примечание. Снайпер сеансов PMON может автоматически прерывать сеансы по истечении заданного времени простоя. Также это можно делать с помощью профилей или диспетчера ресурсов.

Устранение конфликтов блокировок с помощью SQL

Чтобы выявить и прервать заблокированный сеанс, можно использовать операторы SQL.

1
```
SQL> select sid, serial#, username
     from v$session where sid in
     (select blocking_session from v$session)
```

Результат:

SID	SERIAL#	USERNAME
144	8982	HR

2
```
SQL> alter system kill session '144,8982' immediate;
```

ORACLE

© Oracle, 2007. Все права защищены.

Устранение конфликтов блокировок с помощью SQL

Управление сеансами, как и большинство других заданий, можно осуществлять не только в Enterprise Manager, но и с помощью операторов SQL. Таблица `v$session` содержит подробные сведения обо всех подключенных сеансах. `blocking_session` – идентификатор блокирующего сеанса. Если запросить `SID` и `SERIAL#` (где `SID` совпадает с идентификатором заблокированного сеанса), вы получите всю необходимую информацию для выполнения операции `kill session` (удаления сеанса).

Примечание. Для автоматического отключения сеансов, которые бездействуют и блокируют другие сеансы, можно использовать диспетчер ресурсов базы данных (Database Resource Manager).

Взаимные блокировки

Транзакция 1		Транзакция 2

`UPDATE employees`
`SET salary = salary x 1.1`
`WHERE employee_id = 1000;`
	9:00	`UPDATE employees`
		`SET manager  = 1342`
		`WHERE employee_id = 2000;`

`UPDATE employees`
`SET salary = salary x 1.1`
`WHERE employee_id = 2000;`
	9:15	`UPDATE employees`
		`SET manager  = 1342`
		`WHERE employee_id = 1000;`

`ORA-00060:` `Обнаружена взаимная блокировка во время ожидания ресурса`	9:16	

ORACLE

© Oracle, 2007. Все права защищены.

Взаимные блокировки

Взаимная блокировка – это особый случай конфликта блокировок. Взаимные блокировки возникают при участии двух (или больше) сеансов, когда сеансы ожидают данные, взаимно заблокированные друг другом. Так как каждый из сеансов ожидает данные, заблокированные противоположной стороной, ни один из сеансов не может завершить транзакцию и устранить конфликт.

База данных Oracle автоматически выявляет взаимные блокировки и прерывает выполнение оператора с сообщением об ошибке. Реакцией на ошибку должна быть операция подтверждения или отмены изменений, которая освободит блокировку одного из сеансов, благодаря чему второй сеанс сможет завершить свою транзакцию.

В примере, приведенном в данном слайде, в ответ на ошибку о возникновении взаимной блокировки транзакция 1 должна выполнить операцию подтверждения или отмены изменений. В случае подтверждения потребуется еще раз выполнить второе обновление, чтобы успешно завершить транзакцию. В случае отката потребуется выполнить оба обновления, чтобы успешно завершить транзакцию.

Сводка

В этом уроке вы научились:

- управлять данными с помощью SQL;
- идентифицировать и администрировать объекты PL/SQL;
- описывать триггеры и триггерные события;
- отслеживать и устранять конфликты блокировок.

ORACLE

© Oracle, 2007. Все права защищены.

Обзор упражнения 9: Управление данными и параллельным доступом

В данном упражнении рассматриваются следующие темы:

- выявление конфликтов блокировок;
- устранение конфликтов блокировок.

ORACLE

© Oracle, 2007. Все права защищены.